EMBEDDED SYSTEM DESIGN EXPERIMENT

嵌入式系统设计实验

曾毓

实验 07 GUI应用实验

杭州電子科技大学
HANGZHOU DIANZI UNIVERSITY

OLED液晶显示模块简介

OLED显示屏目前被广泛应用于移动设备甚至电视上。其优点在于可视角度大、响应速度快、体积较为轻薄，不过售价偏高，寿命较低。本次实验采用的是市面上较为常见的0.96寸单色OLED显示屏，分辨率为128×64点阵，实物如下图所示。

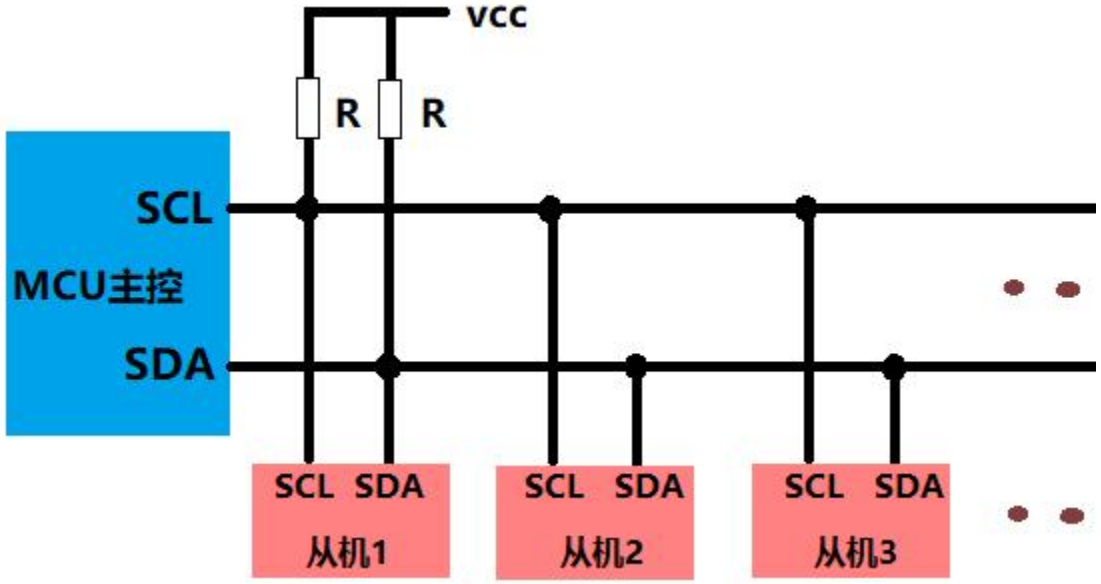
这种OLED屏的内部驱动IC为SSD1306，通信方式一般为SPI或者IIC总线。图示为IIC接口的OLED显示屏，接入学习板时注意引脚电源位置（可能引脚排列不同）。

OLED屏朝外
VCC在右下方

IIC总线简介

IIC（I2C）属于两线式串行总线，由飞利浦公司开发用于微控制器（MCU）和外围设备（从设备）进行通信的一种总线，属于**一主多从**式的总线结构，总线上的每个设备都有一个特定的设备地址，以区分同一总线上的其他设备。

物理IIC接口有两根双向线，串行时钟线（SCL）和串行数据线（SDA）组成，可用于发送和接收数据，但是通信都是由主设备发起，从设备被动响应，实现数据的传输。



IIC通信实现

1. 使用IIC控制器实现，即使用MCU的IIC外设，也就是硬件IIC。该方法使用相应的IIC驱动电路和专用的IIC引脚，通信效率更高，编写代码会相对简单。通信时不需要用代码去控制SCL、SDA的各种高低电平变化来实现IIC协议，只要调用IIC的控制函数，将IIC协议中的可变部分（如：从设备地址、传输数据等等）通过函数传参给控制器，控制器就会自动按照IIC协议实现传输。

2. 使用GPIO通过软件模拟实现。如果芯片上没有IIC控制器，或者控制接口不够用，可以使用任意IO端口模拟实现IIC通信协议。程序设计时通过编写代码控制IO端口的电平变化，模拟IIC协议的时序，来实现IIC的信号和数据传输。

相对硬件IIC，软件模拟IIC更为灵活，受限制更少，在不需要高速IIC通信时，采用软件模拟IIC的应用场景更多。为方便移植，本次实验的OLED应用示例也将使用软件IIC方法。

IIC接口液晶显示模块驱动简介

实验课程的资料→模块驱动下载中的OLED显示屏驱动，文件包名为oled.rar，解开该压缩文件后如下左图得到一个名为OLED的文件夹，里面有7个文件。

OLED显示驱动的7个文件中，首先需要注意的是**SSD1306_i2c.c**源文件中的引脚定义。如下右图所示，该文件的开头定义了三个宏分别对应两个引脚及其相应的端口（一般默认两个引脚都是同一个端口）。如果OLED显示屏接到学习板上的其它引脚，要注意图中的这两个引脚名称是否对应真正所用的引脚。

F08_ASCII.c 类型: C 文件	1	默认英文点阵字库源文件	修改日期: 2017/11/29 12:04 大小: 18.8 KB
GUI.h 类型: H 文件	2	图形库头文件	修改日期: 2025/12/2 13:33 大小: 17.2 KB
GUISlim.c 类型: C 文件	3	图形库源文件	修改日期: 2025/12/2 15:42 大小: 39.0 KB
ssd1306.c 类型: C 文件	4	OLED显示驱动源文件	修改日期: 2024/11/19 19:43 大小: 10.4 KB
ssd1306.h 类型: H 文件	5	OLED显示驱动头文件	修改日期: 2025/12/2 13:15 大小: 1.41 KB
ssd1306_i2c.c 类型: C 文件	6	OLED引脚I2C接口源文件	修改日期: 2024/11/19 19:12 大小: 1.87 KB
ssd1306_i2c.h 类型: H 文件	7	OLED引进I2C接口头文件	修改日期: 2024/11/19 19:12 大小: 316 字节

ssd1306_i2c.c

```
#include "ssd1306_i2c.h"
#include "main.h"

#define OLED_SCL     I2C_SCL_Pin
#define OLED_SDA     I2C_SDA_Pin
#define OLED_GPIO    I2C_SCL_GPIO_Port

```

IIC接口液晶显示模块驱动简介

OLED驱动中的SSD1306.c文件，提供了OLED屏幕初始化、写命令、绘图和刷新等一系列驱动函数。OLED屏幕初始化时，驱动程序为倒放的屏幕做了旋转处理，如果需要正向显示，可以如下修改**SSD1306.c**源文件中的第57行处代码。

```
void SSD1306_init() {
  HAL_Delay(100);
  SSD1306_WriteByte(0xAE,OLED_CMD);
  ...

  SSD1306_WriteByte(0xFF,OLED_CMD);
  SSD1306_WriteByte(0xA0,OLED_CMD); // 上下翻转 正向显示时改为0xA1
  SSD1306_WriteByte(0xC0,OLED_CMD); // 左右翻转 正向显示时注释该行
  SSD1306_WriteByte(0xA6,OLED_CMD);
  SSD1306_WriteByte(0xA8,OLED_CMD);
  SSD1306_WriteByte(0x3F,OLED_CMD);
  ...
```

GUISlim图形库介绍

OLED驱动文件中的GUISlim.c是GUISlim图形库的源码文件，相应的GUI.h头文件中，提供了图形库初始化、绘图、刷新和相关数据结构的声明。

GUISlim图形库是从ucGUI源码中截取的一小部分功能，将其裁剪适配OLED单色显示后，可提供基本的2D绘图、点阵文字和单色图形显示功能。

GUISlim图形库的使用方法也比较简单，操作步骤一般如下：

1. 将OLED驱动程序目录添加到MDK工程后，设置好工程包含路径、注意OLED引脚对应的名称定义。

2. 在freertos.c文件中添加包含gui.h头文件，在任务循环之前调用**GUI_Init**函数初始化图形库。

3. 先使用绘图函数绘制GUI内容，最后调用**GUI_Update**函数刷新屏幕，把绘制的内容显示到OLED屏幕上。

GUISlim图形库介绍

图形库初始化函数:

int GUI_Init(void); **// 图形库初始化，包括OLED屏幕初始化动作**

绘图函数:

void GUI_Clear(void); **// 清屏函数，用背景色填满屏幕**

void GUI_ClearRect(int x0, int y0, int x1, int y1); // 区域擦除函数，用背景色填满指定的矩形区域

void GUI_DrawBitmap(const GUI_BITMAP GUI_UNI_PTR * pBM, int x0, int y0); // 显示图片

void GUI_DrawLine(int x0, int y0, int x1, int y1); // 绘制任意端点的线段

void GUI_DrawHLine(int y0, int x0, int x1); // 绘制水平线

void GUI_DrawVLine(int x0, int y0, int y1); // 绘制垂直线

void GUI_DrawPixel(int x, int y); // 用前景色在指定位置打点

void GUI_DrawRect(int x0, int y0, int x1, int y1); // 画矩形框函数，填充矩形框用GUI_FillRect函数

void GUI_DrawCircle(int x0, int y0, int r); // 画圆函数，填充圆用GUI_FillCircle函数

void GUI_DrawEllipse(int x0, int y0, int rx, int ry); // 画椭圆函数，填充椭圆用GUI_FillEllipse函数

刷新函数:

void GUI_Update(void); **// 屏幕显示刷新，把显示缓冲数据写入OLED屏幕**

GUISlim图形库介绍

显示文本函数:

GUI_FONT GUI_UNI_PTR * GUI_SetFont(const GUI_FONT GUI_UNI_PTR * pNewFont); // 设置字体

void GUI_DispChar (U16 c); // 显示字符函数，在当前位置显示一个字符

void GUI_DispCharAt(U16 c, I16P x, I16P y); // 显示字符函数，在指定位置显示一个字符

void GUI_DispString (const char GUI_UNI_PTR *s); // 从当前位置开始显示一个字符串

void GUI_DispStringAt(const char GUI_UNI_PTR *s, int x, int y); // 从指定位置开始显示一个字符串

void GUI_DispStringHCenterAt(const char GUI_UNI_PTR *s, int x, int y); // 在指定位置居中显示字符串

void GUI_DispStringInRect(const char GUI_UNI_PTR *s, GUI_RECT* pRect, int Flags); // 在指定矩形框中显示一个字符串

int GUI_SetTextAlign(int Align); // 设置文本对齐格式，有靠左、靠右、居中等几种格式

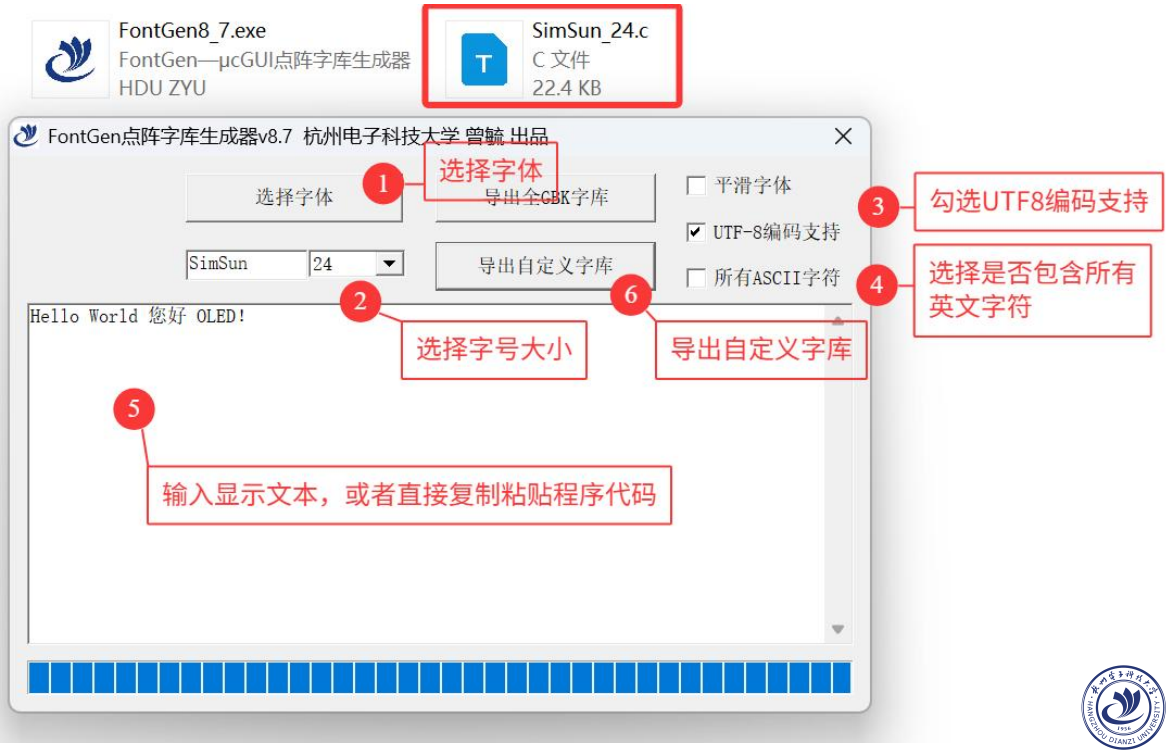
int GUI_GetStringDistX(const char GUI_UNI_PTR *s); // 获取显示待字符串的横向宽度（像素大小）

字体结构体数据类型GUI_FONT和字体文件结构，参见教材8.1.4小节内容

汉字点阵文件制作

开发人员要生成自定义的点阵字库时，可以使用CubeMX安装固件包中的emWin字库生成工具软件FontCvtST.exe，或者使用课程提供的字库生成软件**FontGen8_7**.exe。如下图所示，将需要显示的文本复制粘贴到FontGen程序的文本输入框中，选择好字体类型、大小高度以及是否导出所有ASCII字符（建议勾选），然后单击“导出自定义字库”就能生成指定的字库文件。

注意，要支持utf8编码中文，需下载更新FontGen程序到8_7以上版本。



点阵图库文件生成

在CubeMX安装的固件包中，还有一个名为BmpCvtST.exe的工具软件。要生成一个单色的点阵图库文件，先用BmpCvtST程序打开图片。

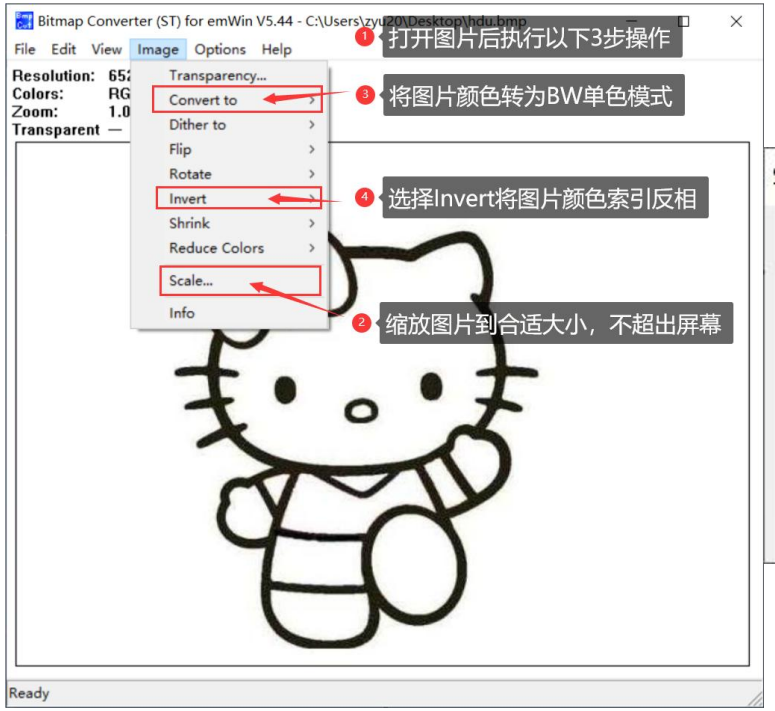
1. 选择菜单“Image”→“Scale...”，如右上图弹出窗口中设置Width和Height，不超过OLED屏幕大小（128x64），单击“OK”按钮确定缩放。

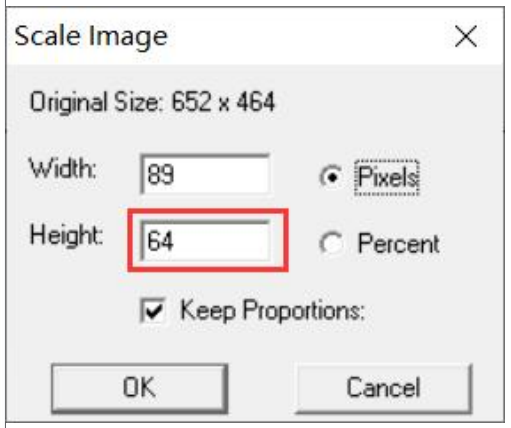
2. 选择菜单“Image”→“Convert to”→“BW”，转换图片为黑白颜色模式。

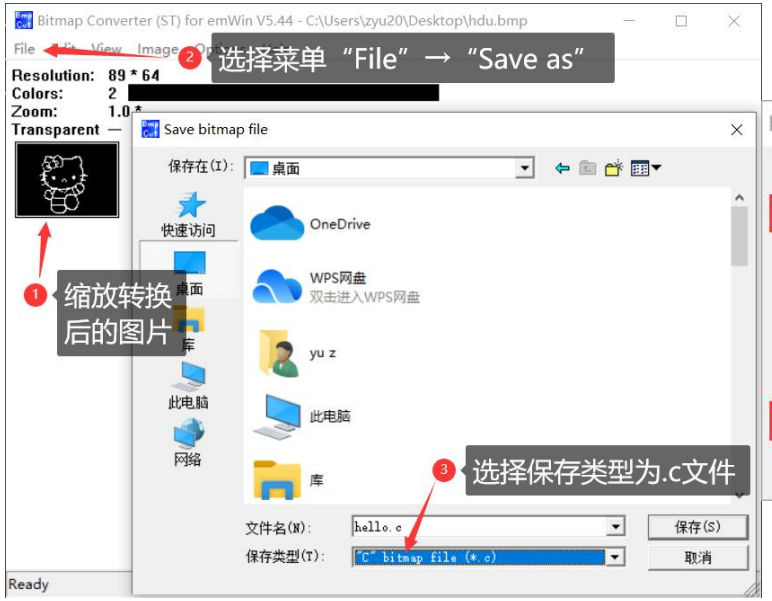
3. 如果所选图片是白底黑字，那么还需要选择菜单“Image”→“Invert”→“Indices”将图片索引反相。

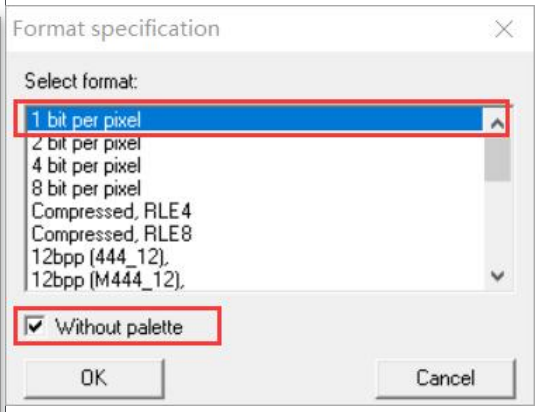
4. 最后，选择菜单“File”→“Save as...”导出图片点阵（.c）文件，如右图所示。

5. 导出格式选项如右下图选择每个点1位，并且不带调色板。





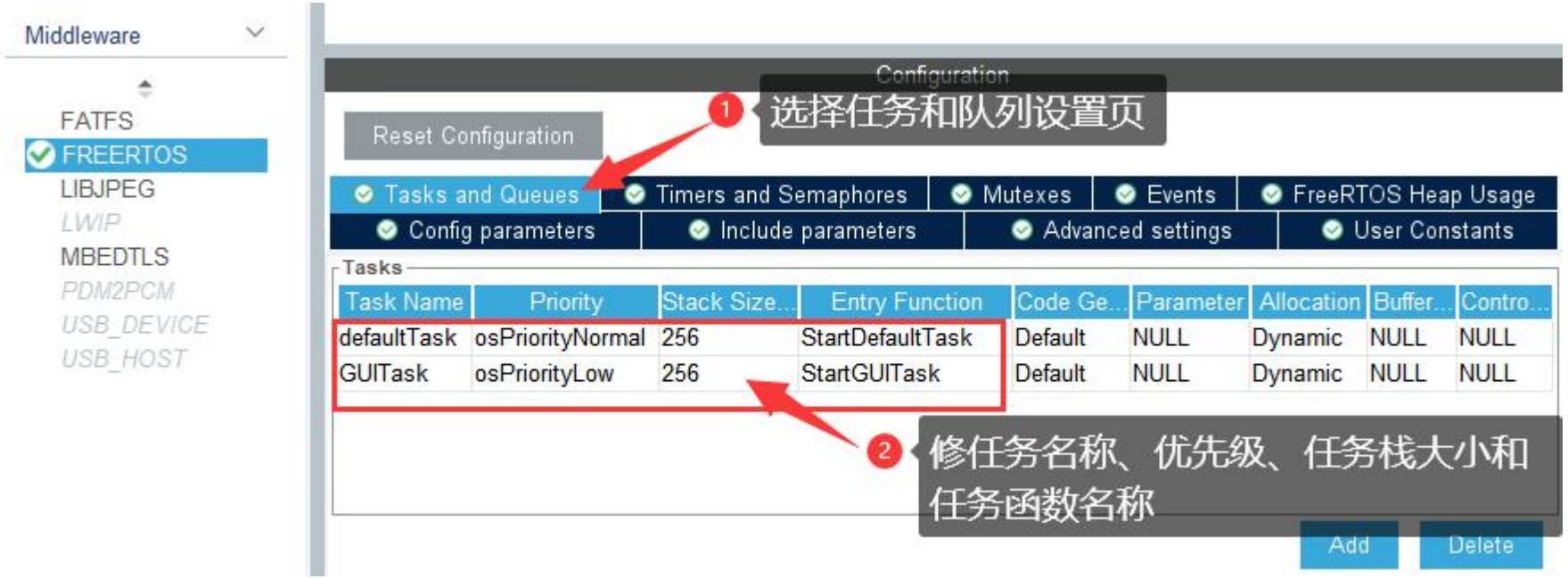




OLED应用实践——修改CubeMX工程

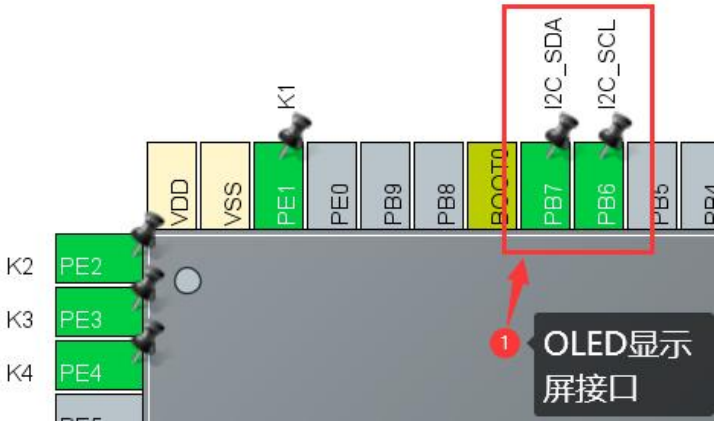
接下来将演示学习板连接OLED屏进行文字和图像显示的一个简单示例。复制实验4的EX04工程文件夹，将复制文件夹重命名为EX07，并将该文件夹中的.ico文件重命名为EX07.ico，将EX07中已有的MDK-ARM子目录删除，准备重新生成STM32工程。

双击EX07.ico文件启动STM32CubeMX打开Cube工程，修改FREERTOS任务属性如下所示。

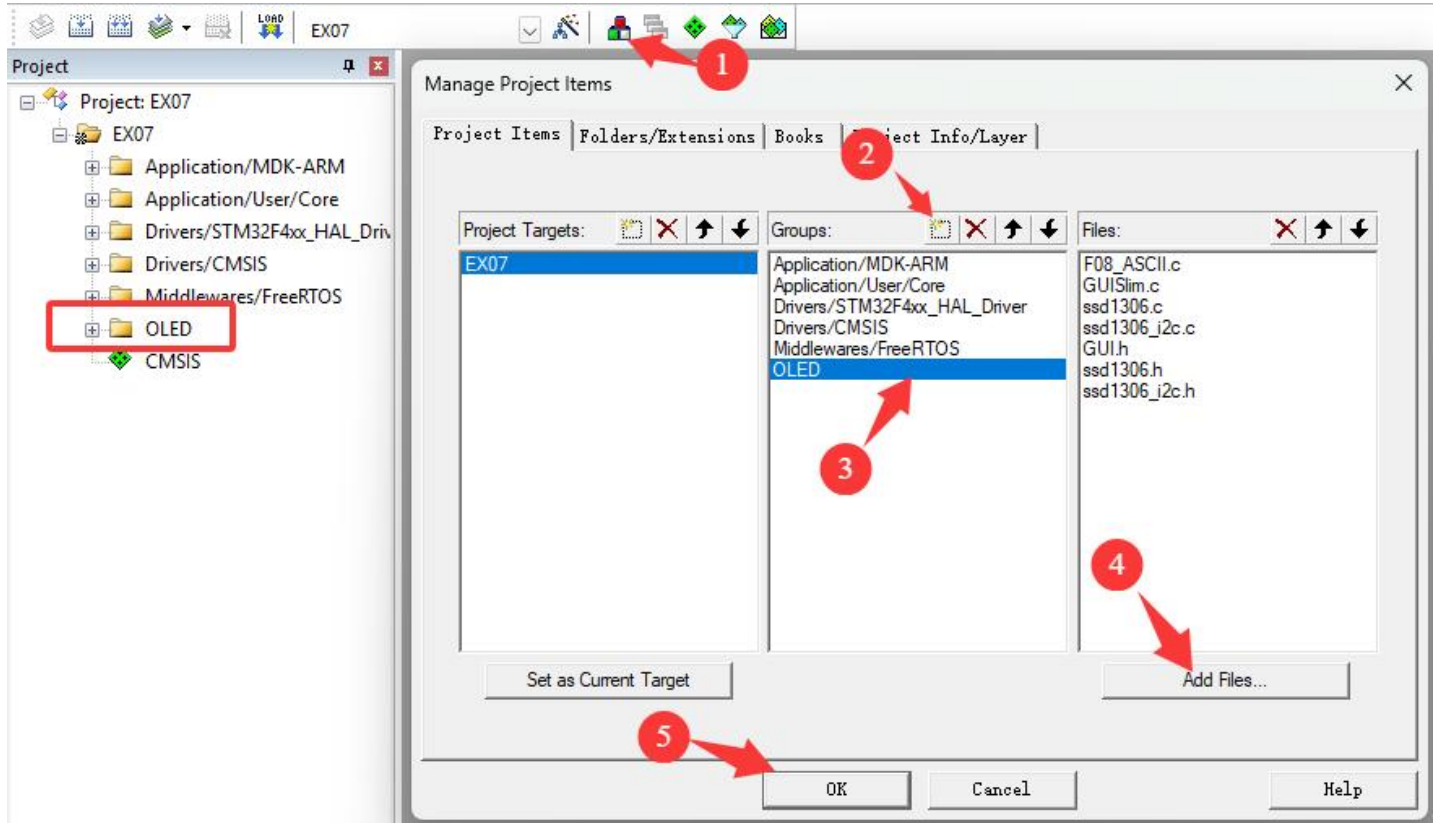


OLED应用实践——添加OLED驱动

回到CubeMX的Pinout & Configuration标签页，添加OLED屏接口引脚。如右所示，添加**PB6、PB7两个输出**端口，将其标签名称分别命名为**I2C_SCL**和**I2C_SDA**。

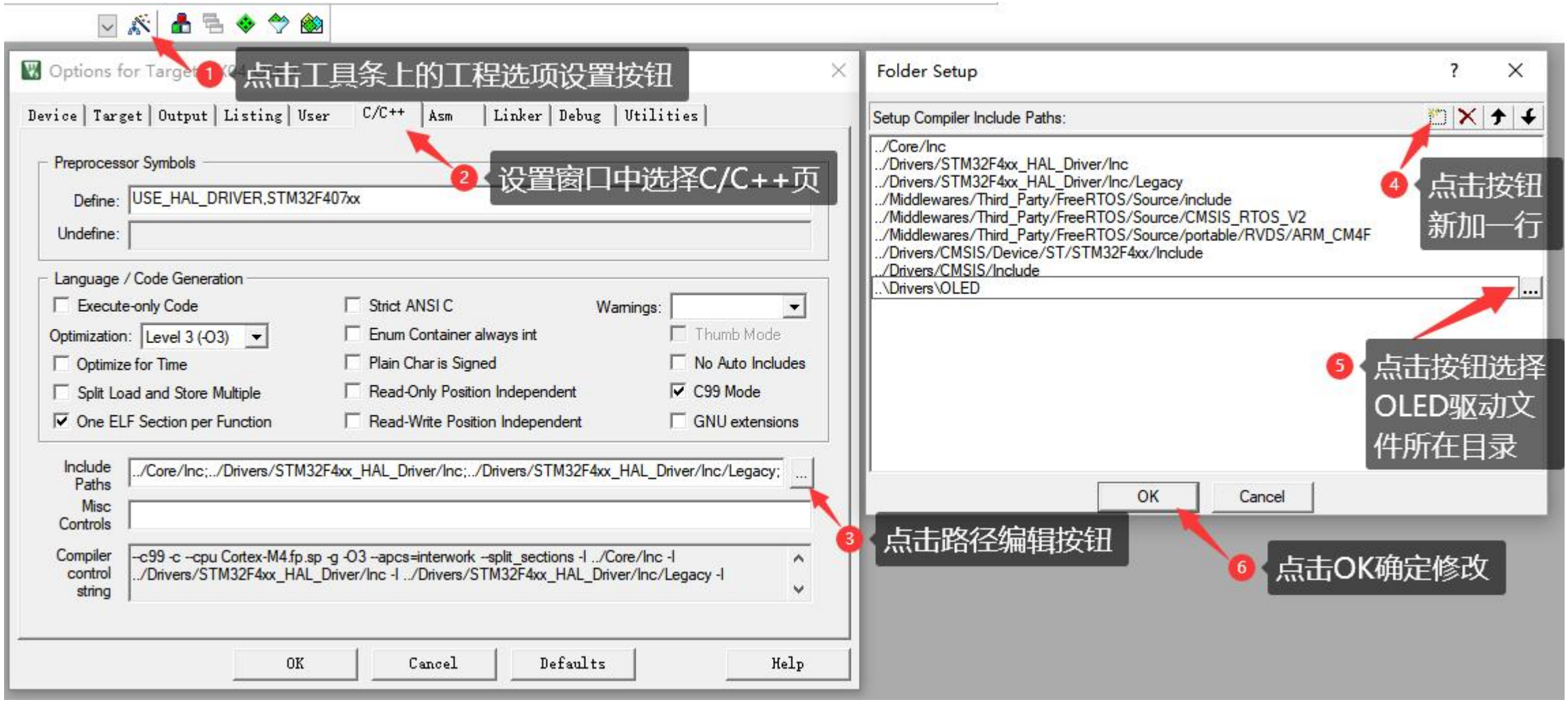


重新导出生成EX07的MDK工程，**复制OLED文件夹**到EX07工程的**Drivers**目录下。用MDK打开工程后，添加OLED文件组并将OLED驱动程序文件到OLED文件组中，操作如右图所示。



OLED应用实践——设置工程包含路径

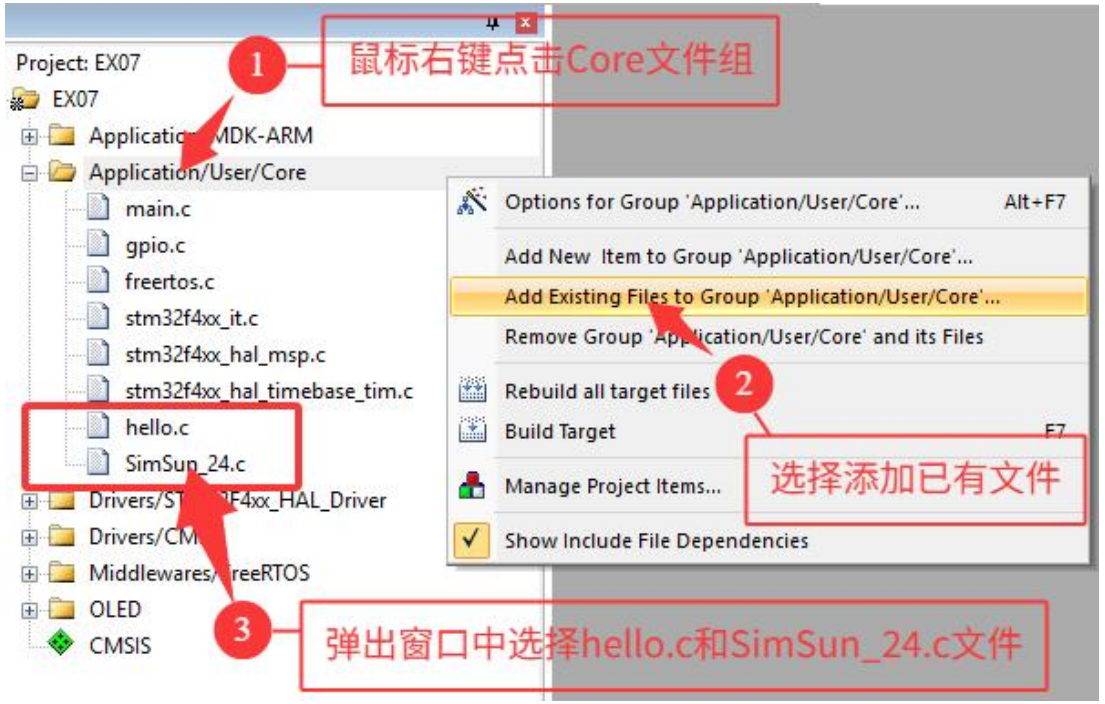
设置工程包含路径添加OLED驱动文件所在路径，操作如下图所示。

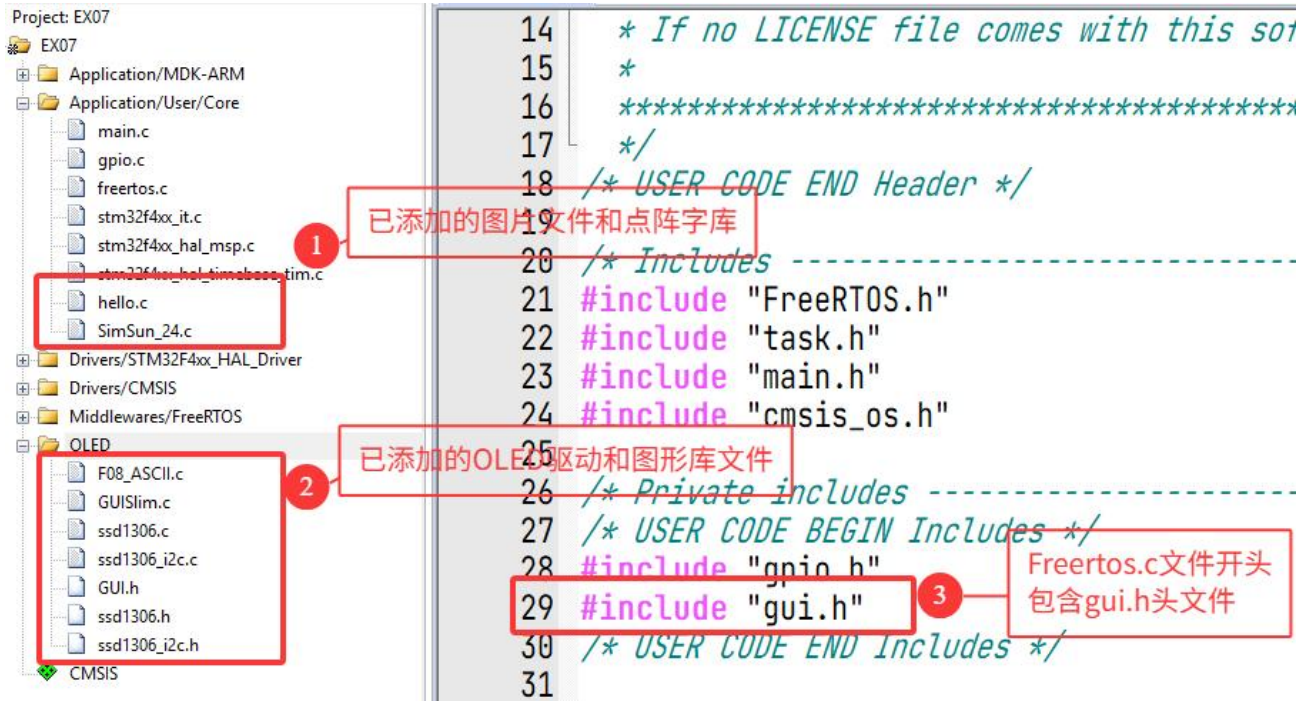


OLED应用实践——添加图片和点阵字库文件

参照前述第10、11页的方法，生成SimSun_24.c点阵图片文件和hello.c点阵字库文件，将两个文件复制到EX07工程的Core/Src子目录中。然后在MDK工程中添加这两个文件到“Application/User/Core”文件组。

最后，在freertos.c文件开头包含gui.h头文件，编译工程，检查是否有编译错误。





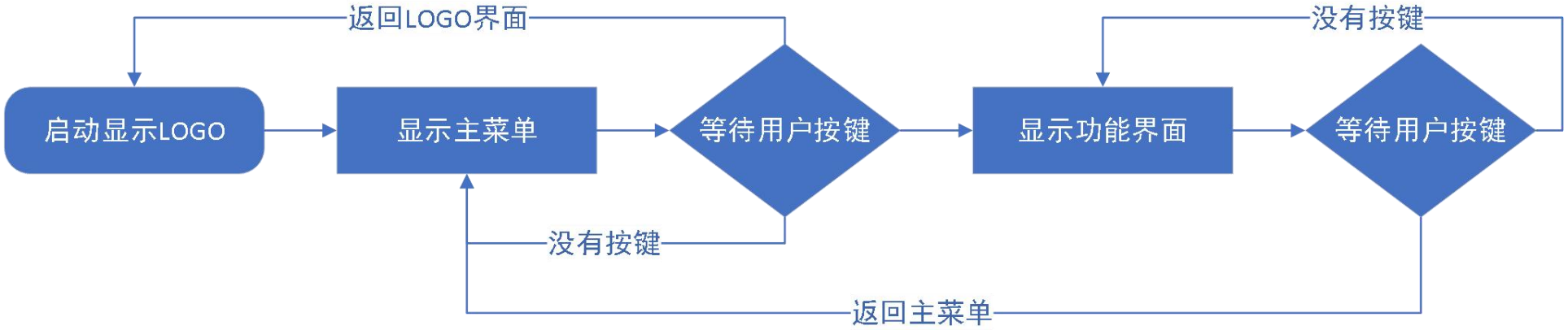
OLED应用实践——添加文字和图片显示功能

为了在freertos.c文件中方便引用SimSun_24字体和hello图片，在MDK中打开SimSun_24.c文件和hello.c文件，将两个文件开头的extern语句复制粘贴到freertos.c文件的变量定义区，再修改GUI任务函数，添加图片和文字显示代码如下。最后编译、下载测试结果如右图所示。

```
/* USER CODE BEGIN Variables */
extern GUI_CONST_STORAGE GUI_BITMAP bmhello;
extern GUI_FLASH const GUI_FONT GUI_FontHZ_SimSun_24;
/* USER CODE END Variables */
...
void StartGUITask(void *argument) {
  /* USER CODE BEGIN StartGUITask */
        GUI_Init();                                                          // 初始化OLED屏和GUI图形库
        GUI_SetFont(&GUI_FontHZ_SimSun_24);      GUI_DispString("您好！\nOLED"); // 选用24号宋体，显示文字
        GUI_DrawBitmap(&bmhello, 128 - bmhello.XSize, 0);                    // 靠右显示一张图片（64x64大小）
        GUI_SetFont(GUI_DEFAULT_FONT);                                       // 选用默认6x8英文字体
  for(;;) {                                                                  // 任务循环
        char str[20]; sprintf(str, "%d", osKernelGetTickCount());            // 格式化字符串，准备显示当前系统时间戳
        GUI_DispStringAt(str, 0, 64 - GUI_GetFontSizeY());                   // 左下角显示字符串
        GUI_Update();          osDelay(10);                                  // 刷新屏幕显示、延时10毫秒
  }
  /* USER CODE END StartGUITask */
}
```

多级菜单GUI程序结构

在GUI界面比较多的时候，通常使用多级菜单的形式来设计GUI程序界面。例如程序上电启动后显示logo，然后进入到GUI任务主循环中，默认显示功能主菜单页面，主菜单页面中显示有不同的功能项，通过按键可以切换进入某个功能项，在功能项界面中，用户又可以通过不同的按键进入功能子项界面或者返回主菜单界面。



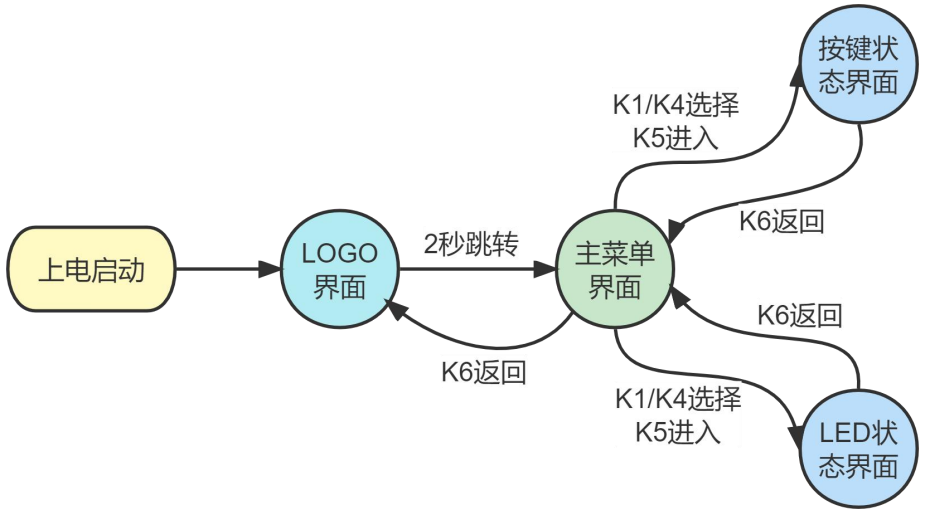
程序设计时，可以设计一个有限状态机的程序结构，定义一个整数或枚举变量来表示屏幕当前显示的界面，然后在按键任务中对应不同的按键和界面状态来修改变量的值。

在GUI任务主循环中使用switch语句对该变量进行分支判断，不同的界面调用不同的界面显示子函数。

简单GUI设计示例——UI功能设计

基于之前的EX07工程，本次实验设计一个简单的GUI应用，程序包含4个界面，功能分配如下：

1. LOGO界面，系统上电启动后显示校徽图标，保持2秒后自动跳转到主菜单。
2. 主菜单界面，显示应用标题和菜单项（LED状态和按键状态），可用按键K1、K4上下选择待选项，按键K5进入待选项界面，按键K6返回LOGO界面。
3. LED状态界面，显示当前8个LED灯的亮灭状态，按键K6返回主菜单界面。
4. 按键状态界面，显示按键K1 ~ K4的按压状态，按键K6返回主菜单界面。

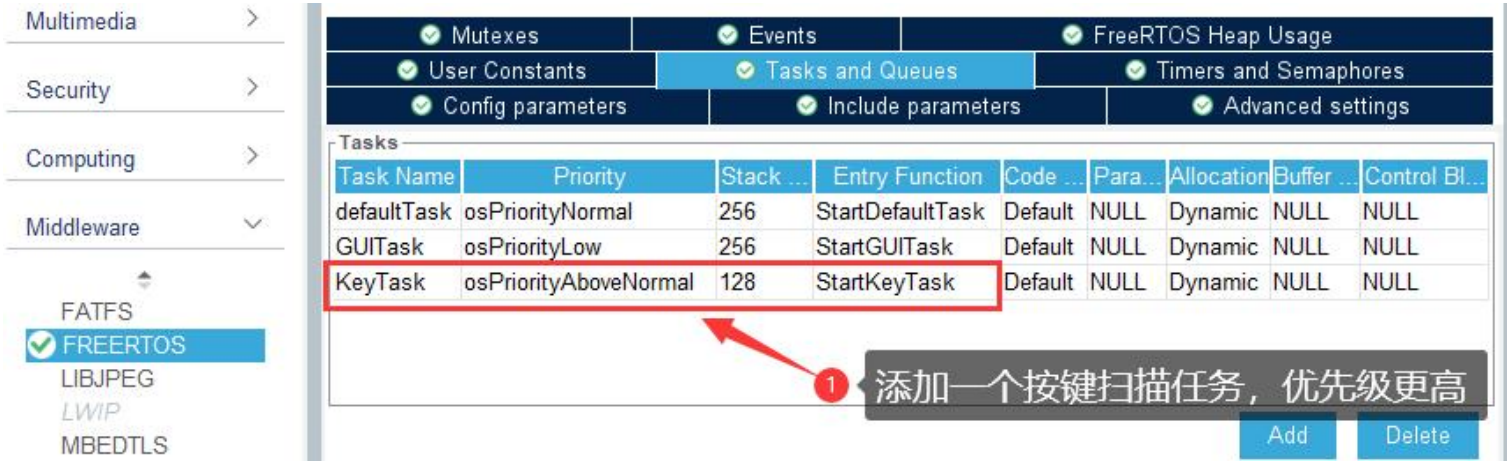


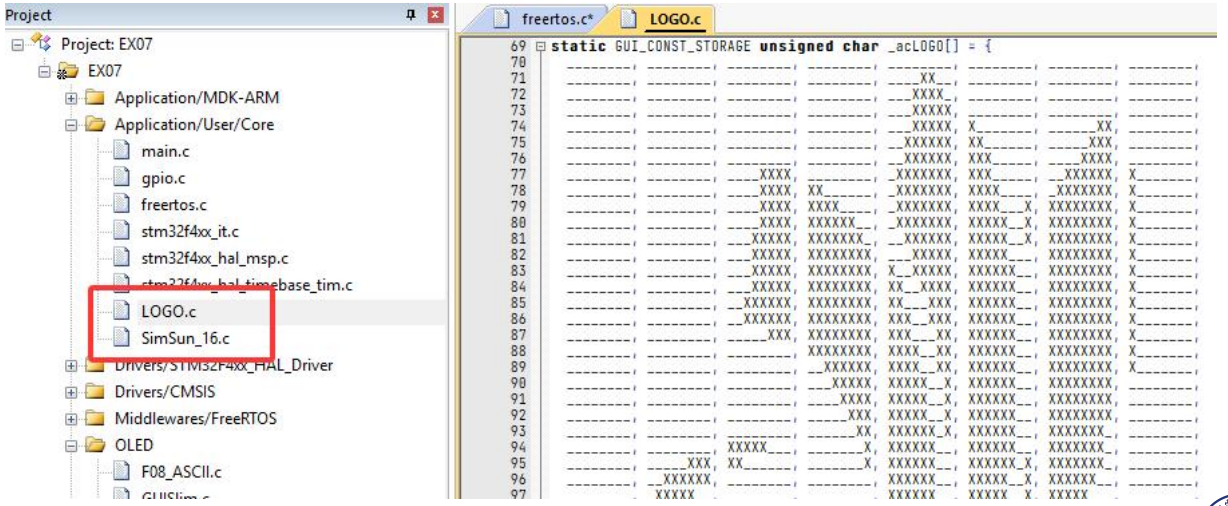
简单GUI设计示例——修改CubeMX工程、生成字体、图片资源文件

根据以上应用需求，在FreeRTOS设置中**添加按键扫描任务**，完成设置之后，再次导出生成EX07的MDK工程。

同时，利用前述GUI工具软件生成校徽点阵图片文件LOGO.c和宋体16号点阵字体文件SimSun_16.c，包含“主菜单、状态、LED灯、按键、○●”等文字。将这两个文件添加到MDK工程。

删除之前添加的hello.c和SimSun_24.c文件，结果如右：





简单GUI设计示例——修改gpio.c按键扫描驱动

本次实验要用到6个按键，先确认一下在CubeMX中添加了K5、K6两个按键连接的单片机端口，然后打开gpio.c文件，确认ScanKey函数也已经包含了K5、K6两个按键的扫描动作，代码如下：

```
uint16_t ScanKey(void) {
    uint16_t key = 0;// 按键按下时，K1 ~ K4低电平有效，K5、K6高电平有效
    if (HAL_GPIO_ReadPin(K1_GPIO_Port, K1_Pin) == GPIO_PIN_RESET)      key |= K1_Pin;
    if (HAL_GPIO_ReadPin(K2_GPIO_Port, K2_Pin) == GPIO_PIN_RESET)      key |= K2_Pin;
    if (HAL_GPIO_ReadPin(K3_GPIO_Port, K3_Pin) == GPIO_PIN_RESET)      key |= K3_Pin;
    if (HAL_GPIO_ReadPin(K4_GPIO_Port, K4_Pin) == GPIO_PIN_RESET)      key |= K4_Pin;
    if (HAL_GPIO_ReadPin(K5_GPIO_Port, K5_Pin) == GPIO_PIN_SET)        key |= K5_Pin;
    if (HAL_GPIO_ReadPin(K6_GPIO_Port, K6_Pin) == GPIO_PIN_SET)        key |= K6_Pin;
    if (key > 0) { osDelay(10);        // 按键延时10毫秒消抖
        uint16_t key2 = 0;
        if (HAL_GPIO_ReadPin(K1_GPIO_Port, K1_Pin) == GPIO_PIN_RESET) key2 |= K1_Pin;
        if (HAL_GPIO_ReadPin(K2_GPIO_Port, K2_Pin) == GPIO_PIN_RESET) key2 |= K2_Pin;
        if (HAL_GPIO_ReadPin(K3_GPIO_Port, K3_Pin) == GPIO_PIN_RESET) key2 |= K3_Pin;
        if (HAL_GPIO_ReadPin(K4_GPIO_Port, K4_Pin) == GPIO_PIN_RESET) key2 |= K4_Pin;
        if (HAL_GPIO_ReadPin(K5_GPIO_Port, K5_Pin) == GPIO_PIN_SET)        key2 |= K5_Pin;
        if (HAL_GPIO_ReadPin(K6_GPIO_Port, K6_Pin) == GPIO_PIN_SET)        key2 |= K6_Pin;
        if (key == key2)       return key;            // 返回有效按键
    }
    return 0;
}
```

简单GUI设计示例——freertos.c修改全局变量

最后回到freertos.c文件，修改按键组合掩码、全局变量和外部变量声明如下：

```
/* Private macro -------------------------------------------------------------*/
/* USER CODE BEGIN PM */
// 定义6个按键组合掩码
#define KEY_MASK  (K1_Pin | K2_Pin | K3_Pin | K4_Pin | K5_Pin | K6_Pin)
/* USER CODE END PM */

/* Private variables ---------------------------------------------------------*/
/* USER CODE BEGIN Variables */
typedef enum {GUI_LOGO, GUI_MAIN, GUI_LED, GUI_KEY} GUI_STATE;        // 用枚举类型表示UI界面状态
GUI_STATE g_sta = GUI_LOGO;                    // 当前UI界面
GUI_STATE g_sta_select = GUI_LED;              // 代进入的UI界面

extern GUI_CONST_STORAGE GUI_BITMAP bmLOGO;                // LOGO界面校徽图片
extern GUI_FLASH const GUI_FONT GUI_FontHZ_SimSun_16;      // 16号宋体点阵字体
/* USER CODE END Variables */
```

简单GUI设计示例——freertos.c修改默认流水灯任务

```
void StartDefaultTask(void *argument) {
  /* USER CODE BEGIN StartDefaultTask */
  uint8_t sta = 0x01;          // 流水灯初始状态（L1亮）
  uint8_t dir = 1;             // 流水灯初始方向（1：右行，0：左行）
  for(;;) {
    SetLeds(sta);      // 流水灯亮灯
    // 等待按键消息
    int key = osThreadFlagsWait(KEY_MASK, osFlagsWaitAny, 0);
    if (key == K1_Pin)         // K1按下
              dir = !dir;
    if (dir)    sta = (sta < 0x80) ? (sta << 1) : 0x01;
    else       sta = (sta > 0x01) ? (sta >> 1) : 0x80;
    osDelay(100);
  }
  /* USER CODE END StartDefaultTask */
}
```

简单GUI设计示例——freertos.c修改默认按键扫描任务

```
void StartKeyTask(void *argument) {
  /* USER CODE BEGIN StartKeyTask */
  for(;;) {
    uint16_t key = ScanKey();     // 扫描按键键码
    if (key > 0) {            // 有按键按下
      while (ScanKey() > 0);                    // 等待按键放开，防止按键连按
      osThreadFlagsSet(defaultTaskHandle, key); // 向默认任务发送key通知
    }
    switch (key) {
        default:        break;
        case K1_Pin:  if (GUI_MAIN == g_sta)        g_sta_select = GUI_LED;    break;
        case K4_Pin:  if (GUI_MAIN == g_sta)        g_sta_select = GUI_KEY;    break;
        case K5_Pin:  if (GUI_MAIN == g_sta)        g_sta = g_sta_select;      break;
        case K6_Pin:  if (GUI_MAIN == g_sta)        g_sta = GUI_LOGO;
                      else                          g_sta = GUI_MAIN;
                      break;
    }
    osDelay(10);
  }
  /* USER CODE END StartKeyTask */
}
```

简单GUI设计示例——freertos.c修改GUI显示任务

```
void StartGUITask(void *argument) {
  /* USER CODE BEGIN StartGUITask */
  GUI_Init();// OLED初始化、GUI图形库初始化

  for(;;) {
    switch (g_sta) {            // 根据当前显示状态调用对应绘图函数
            default: break;
            case GUI_LOGO: UILogo();        break;
            case GUI_MAIN: UIMain();        break;
            case GUI_LED:    UILeds();      break;
            case GUI_KEY:    UIKeys();      break;
    }
    osDelay(10);
  }
  /* USER CODE END StartGUITask */
}
```

简单GUI设计示例——freertos.c添加LOGO界面绘图函数

```
/* USER CODE BEGIN Application */
void UILogo(void) {
    static uint32_t tick = 0;  // 定义静态变量，存储进入界面时的时间戳
    if (0 == tick)     tick = osKernelGetTickCount();  // 开始进入界面时，记录时间戳

    GUI_Clear();                    // 屏幕内容清空
    GUI_DrawBitmap(&bmLOGO,
            (128 - bmLOGO.XSize) / 2, (64 - bmLOGO.YSize) / 2); // 居中显示校徽图片

    GUI_Update();                   // 刷新屏幕显示
    if (osKernelGetTickCount() >= tick + 2000) {     // 如果当前时间已经超过进入时间2秒
            g_sta = GUI_MAIN;       // 界面状态跳转到主菜单界面
            tick = 0;                               // 时间戳清零，以备再次进入
    }
}
```

简单GUI设计示例——freertos.c添加主菜单界面绘图函数

```
void UIMain(void) {
    GUI_Clear();                    // 屏幕内容清空
    GUI_SetFont(&GUI_FontHZ_SimSun_16);         // 设置文本字体为16号宋体
    GUI_DispStringHCenterAt("主菜单", 64, 0);       // 屏幕正上方居中显示标题
    if (GUI_LED == g_sta_select) {                  // 如果待选界面为LED状态
            GUI_DispStringHCenterAt("* LED状态", 64, 20);
            GUI_DispStringHCenterAt("按键状态", 64, 40);
    }
    else {                                          // 如果待选界面为按键状态
            GUI_DispStringHCenterAt("LED状态", 64, 20);
            GUI_DispStringHCenterAt("* 按键状态", 64, 40);
    }

    GUI_Update();               // 刷新屏幕显示
}
```

简单GUI设计示例——freertos.c添加LED状态界面绘图函数

```
void UILeds(void) {
    GPIO_TypeDef* LED_Ports[8] = {    // 定义LED灯端口数组，如果8个灯都是一个Port，可以不用
            L1_GPIO_Port, L2_GPIO_Port, L3_GPIO_Port, L4_GPIO_Port,
            L5_GPIO_Port, L6_GPIO_Port, L7_GPIO_Port, L8_GPIO_Port};
    uint16_t LED_Pin[8] = {L1_Pin, L2_Pin, L3_Pin, L4_Pin, L5_Pin, L6_Pin, L7_Pin, L8_Pin}; // 引脚数组

    GUI_Clear();                                    // 屏幕内容清空
    GUI_SetFont(&GUI_FontHZ_SimSun_16);             // 设置文本字体为16号宋体
    GUI_DispStringHCenterAt("LED状态", 64, 0); // 屏幕正上方居中显示标题

    int tx, ty;                     // 定义绘图临时变量
    tx = ty = 12;
    for (int i = 0; i < 8; ++i) {   // 循环遍历8个LED灯
            GUI_DrawRect(16 * i + 1, 20, 16 * i + tx, 20 + ty);                // 绘制边框
            if (HAL_GPIO_ReadPin(LED_Ports[i], LED_Pin[i]) == GPIO_PIN_RESET)   // 读取端口电平
                    GUI_FillRect(16 * i + 1, 20, 16 * i + tx, 20 + ty);        // 如果亮灯则填充矩形
    }
    GUI_Update();                   // 刷新屏幕显示
}
```

简单GUI设计示例——freertos.c添加按键状态界面绘图函数

```
void UIKeys(void) {
    GUI_Clear();                                    // 屏幕内容清空
    GUI_SetFont(&GUI_FontHZ_SimSun_16);             // 设置文本字体为16号宋体
    GUI_DispStringHCenterAt("按键状态", 64, 0);// 屏幕正上方居中显示标题

    GPIO_PinState ps = HAL_GPIO_ReadPin(K1_GPIO_Port, K1_Pin);// 定义变量，读取K1按键电平
    GUI_DispStringAt(GPIO_PIN_RESET == ps ? "●" : "○", 26, 20); // 绘制不同符号表示按键按压状态
    ps = HAL_GPIO_ReadPin(K4_GPIO_Port, K4_Pin);                // K4按键
    GUI_DispStringAt(GPIO_PIN_RESET == ps ? "●" : "○", 26, 48);
    ps = HAL_GPIO_ReadPin(K2_GPIO_Port, K2_Pin);                // K2按键
    GUI_DispStringAt(GPIO_PIN_RESET == ps ? "●" : "○", 6, 34);
    ps = HAL_GPIO_ReadPin(K3_GPIO_Port, K3_Pin);                // K3按键
    GUI_DispStringAt(GPIO_PIN_RESET == ps ? "●" : "○", 46, 34);
    ps = HAL_GPIO_ReadPin(K5_GPIO_Port, K5_Pin);                // K5按键
    GUI_DispStringAt(GPIO_PIN_SET == ps ? "●" : "○", 86, 34);   // 注意电平方式和前4个按键不同
    ps = HAL_GPIO_ReadPin(K6_GPIO_Port, K6_Pin);                // K6按键
    GUI_DispStringAt(GPIO_PIN_SET == ps ? "●" : "○", 106, 34);

    GUI_Update();                                   // 刷新屏幕显示
}
/* USER CODE END Application */
```

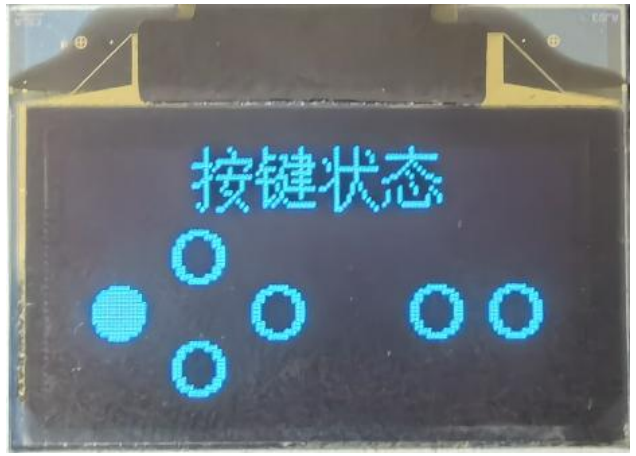
简单GUI设计示例——freertos.c添加页面绘图函数声明

上一页中的4个绘图函数，如果代码添加在GUI任务函数之后，那么需要在freertos.c文件的开头添加这几个函数的声明。

```
/* Private function prototypes -----------------------------------*/
/* USER CODE BEGIN FunctionPrototypes */
void UILogo(void);
void UIMain(void);
void UILeds(void);
void UIKeys(void);
/* USER CODE END FunctionPrototypes */
```

简单GUI设计示例——测试结果

重新编译工程，编译成功后，下载程序到学习板，测试结果如下所示：



实验内容

1. 示例工程实践练习

参照本次实验讲义内容，创建并复现EX07的GUI示例工程，最后在学习板上测试通过。LOGO界面显示校徽改为显示姓名、学号。

2. UI编程练习

在EX07工程基础上添加一个“参数设置”菜单项，选择该菜单项并按K5进入“参数设置”界面，该界面能显示“流水灯速度”和“流水灯方向”两个设置项。能通过K1、K4上下选择修改项，通过K2、K3左右调整流水灯的速度和方向，按K6返回主菜单。

3. 到学习通上的对应课程作业中，完成提交本次实验的**课堂作业**。

思考问题：

为什么流水灯速度快到一定程度时，OLED显示的流水灯状态和实际的LED灯状态不一样了？

下一次实验08

STM32定时器应用实验

请提前做好预习